Custom UDP Server vs. Godot ENet:
A Cloud Benchmark of Multiplayer Server Capacity

Carlos L. Nunez
Department of Computer Science and Engineering, Shippensburg University, Shippensburg, Pennsylvania, USA
Mentor: Dr. Girard, CSC 499 – Senior Research Methods
Fall 2026 (draft)

***Abstract*— Developers who build multiplayer games in Godot must choose between the engine's built-in ENet dedicated server and a custom server written outside the engine. We compare a custom C++ UDP server with Godot's ENet headless server under the same workload, NodeStrike, a 20×20 grid game in which every client is a bot that moves one tile per 20 Hz server tick. Both servers run on Google Cloud with one server VM and three client VMs. Two tests are used: a progressive ladder that adds clients from 1 to 300, run eight times per backend, and a 20-minute hold at fixed loads of 180 to 300 clients. We measure latency, packet handling (snapshots received per client per second), client limit, and server tick rate. TODO: write the results sentences from the granular ladder runs (the hold results are in Sections VIII-B and VIII-C).**

***Index Terms*—**game networking, UDP, ENet, Godot, tick rate, server capacity, benchmarking

I. Introduction

The Godot game engine ships with a built-in multiplayer API built on ENet, a UDP-based reliability library [1], and also exposes PacketPeerUDP for connecting clients to custom servers over raw UDP [2]. ENet provides remote procedure calls, scene synchronization, automatic packet reliability, and connection management out of the box, but its dedicated server runs as a full Godot engine instance [3], carrying overhead that pure networking may not require. A custom C++ UDP server eliminates that overhead and gives the developer full control, at the cost of manually reimplementing what ENet provides by default.

The problem this paper addresses is the lack of a controlled comparison between these two backends in the context of a Godot client. Developers currently rely on assumptions, forum posts, and benchmarks from unrelated engines. We did not find a published study that isolates how Godot's ENet dedicated server performs against a C++ UDP server when both serve the same Godot client under identical workloads.

This paper produces that comparison with three components: a custom C++ UDP server, a headless Godot ENet dedicated server, and a shared Godot client that can connect to either backend. A demo game, NodeStrike, provides a consistent network workload, and an automated benchmarking tool on Google Cloud scales the client count in controlled increments, up to 300 clients. We record and compare latency, packet handling (snapshots received per client per second and skipped snapshots), client capacity, and server tick rate across both backends.

II. Background and Related Work

Multiplayer game networking is rarely visible to the player but plays a critical role in multiplayer communication. Multiplayer development is one of the more demanding areas of game development, requiring careful consideration of latency, scalability, and reliability. Two main options are available to developers building a multiplayer game with the Godot engine: the built-in ENet-based high-level networking system, or a custom UDP server backend.

A. Godot's ENet stack

The Godot engine has a built-in high-level multiplayer API; it is built on top of ENet, a UDP-based networking library. Aukstakalnis describes the resulting ENet-based peer-to-server architecture as one in which "the game players interact with the client, while the server is a stripped-down version that updates the game to the clients" [1, p. 4]. This is essentially a headless server, exported as a version of the Godot project that runs without audio or a graphical interface, leaving a process that runs game logic and relays state updates to clients. While this approach is simple and supports rapid early development, it later introduces architectural constraints, most notably that the server runs as an instance of the engine itself, carrying overhead that may not be needed for pure networking tasks.

Aukstakalnis further notes that ENet provides convenient high-level abstractions such as remote procedure calls (RPCs), automatic packet reliability, and event-controlled callbacks such as on-connect and on-disconnect handlers [1]. According to the Godot Engine documentation on dedicated server exports, this headless mode is officially supported, but the developer must manage the game logic carefully to keep client-only code from running on the server. To handle this, Godot supports flags and arguments such as `--server`, which tell the code to run in server mode [3]. The Godot Engine documentation also describes a different route for developers who want to implement their own server: lower-level networking with raw UDP, TCP, or HTTP [2].

A thesis from the DIVA portal examining server-client networking in Godot summarizes the trade-off well: "Godot offers a high-level multiplayer API to facilitate the creation of multiplayer games in addition to supporting low-level networking via common protocols like UDP, TCP, HTTP, and SSL. Low-level networking is flexible and necessary for custom server implementations, but it usually takes much more time and knowledge to set up" [1, p. 6]. This source provides the baseline that this paper quantifies: the convenience of ENet versus the possible performance gains of a custom low-overhead UDP server written in the mid-level systems language C++.

B. Transport protocol: TCP versus UDP

The selection between the User Datagram Protocol (UDP) and the Transmission Control Protocol (TCP) is fundamental to any discussion of multiplayer networking performance. Al-Dhief et al. provide a comparison and conclude that UDP exhibits significantly lower latency than TCP due to its connectionless, no-order delivery model [4, p. 173]. TCP carries built-in mechanics for packet retransmission, ordering, and congestion control, which retain data integrity and may be valuable for other applications but introduce significant delay in applications that must respond in real time.

In the context of multiplayer games, the reliability that TCP provides is largely unnecessary. Most updates to game state, such as player positions, input events, and world state, are superseded by the next update. A retransmitted packet from a previous frame has no value if it arrives late. UDP is therefore the better choice for the real-time gameplay loop, with sequence numbering layered on top by the application where order matters. Jones reinforces this rationale by stating that UDP's low overhead makes it an ideal choice for fast, real-time applications where latency matters more than guaranteed delivery [5].

C. Implementation language for a custom server

There are many programming languages available for implementing a custom UDP server outside of the Godot engine. Retunsky's benchmarking study of low-level I/O performance across C, C++, Rust, Go, Java, and Python finds that the results can be grouped into two clusters: C, C++, and Rust as high-performance languages, and Rust, Go, Java, and Python as memory-safe languages [6]. This classification emphasizes a fundamental trade-off between raw execution speed and the safety assurances offered by higher-level languages.

Arboleda, Arias, and Riveros add to this analysis by showing that C++ is materially more efficient than Python for parallel, I/O-bound workloads. This finding supports C++ as a strong choice for a high-performance game server expected to handle multiple clients simultaneously [7, p. 8]. Zehra et al. reach a similar conclusion in a memory and time comparison between C++ and Python [8]. There are also many available resources for learning how to set up a C++ UDP server. For example, GeeksforGeeks publishes a tutorial on UDP server-client implementation in C++ that demonstrates the basic socket workflow and indicates that setting up a server with C++ socket libraries is not as difficult as it might first appear, providing a useful comparison point against ENet implementations [9].

D. Network topology

How the network is designed plays an important role in how it functions; network topology in a multiplayer game can affect performance characteristics and scalability. Çelik and Seçinti examine peer-to-peer implementations and client-server topologies in the context of a multiplayer game. Their findings indicate that peer-to-peer implementations carry significant issues with synchronization, cheating, and latency, and that they constrain achievable client limits [10, p. 48]. In a dedicated server topology, or client-server arrangement, all clients communicate independently with the server, which determines the game state. This arrangement is preferred for more competitive multiplayer experiences.

Ahde's thesis on real-time multiplayer server implementation in Unity provides a practical examination of dedicated server design: on one hand a server that handles the logic and processing, and on the other a server that handles the bandwidth and traffic management [11, p. 22]. This paper adopts the dedicated-server topology in its simplest form, with one server process serving all connected clients.

E. Packet loss, buffer bloat, and heartbeats

Latency in a UDP server is not a single number but a distribution that depends on how the server's receive queue is managed under load. When packets arrive faster than the application can drain the socket buffer, queuing delay grows, and eventually new packets are dropped at the kernel level rather than enqueued. Excessive buffering, known as buffer bloat, is a documented cause of high latency under load [12]. Another concern is detecting silent client disconnections; UDP's connectionless nature means the server cannot distinguish between a client that has stopped sending and one whose network has dropped. To address this, the standard architectural mitigation is a heartbeat mechanism: each client transmits a small, periodic packet, and the server evicts any client from which no heartbeat has been received within a predefined, configurable timeout.

F. Summary

The literature reviewed here establishes a foundation for this paper: a comparison of a custom C++ UDP server backend against Godot's built-in ENet dedicated server implementation. The existing body of work confirms that UDP is the appropriate transport protocol for real-time multiplayer games, that C++ offers a measurable performance advantage over higher-level languages for server-side I/O tasks, and that Godot's ENet system, while functional and convenient, carries architectural constraints inherent to its design as a stripped-down game engine instance. Together, these sources provide the theoretical grounding necessary to design, implement, and benchmark both server variants against consistent metrics of latency, packet handling, and client capacity.

G. Ticks and snapshots

TODO: write what a tick and a snapshot are.

TABLE I
Terms used in this paper.

Term	Meaning in NodeStrike
Hz (hertz)	Times per second. NodeStrike runs at 20 Hz, i.e. 20 ticks per second.
Tick	One server cycle: read incoming client updates, update the world, send a snapshot. Fixed at 50 ms ($1000/20$).
Snapshot	A capture of the game state at one tick: every player's position at that instant. The server sends it to each client ($16 + 12N$ bytes for N players, split into datagrams of at most 1,200 bytes). Each arrival updates the client's current ping reading and gap count.
Latency / ping	How long data takes to travel. Round-trip time (RTT) is there and back. In the game client, ping is the time from sending its last position update until the next snapshot arrives, so it includes waiting for the server's next 50 ms tick.
Gap / loss	Each snapshot has a sequence number; a missing number means a snapshot was lost or dropped.
Effective tick rate	Ticks the server actually completes per second. Healthy ≈ 20 Hz; below 18 Hz is treated as overloaded.

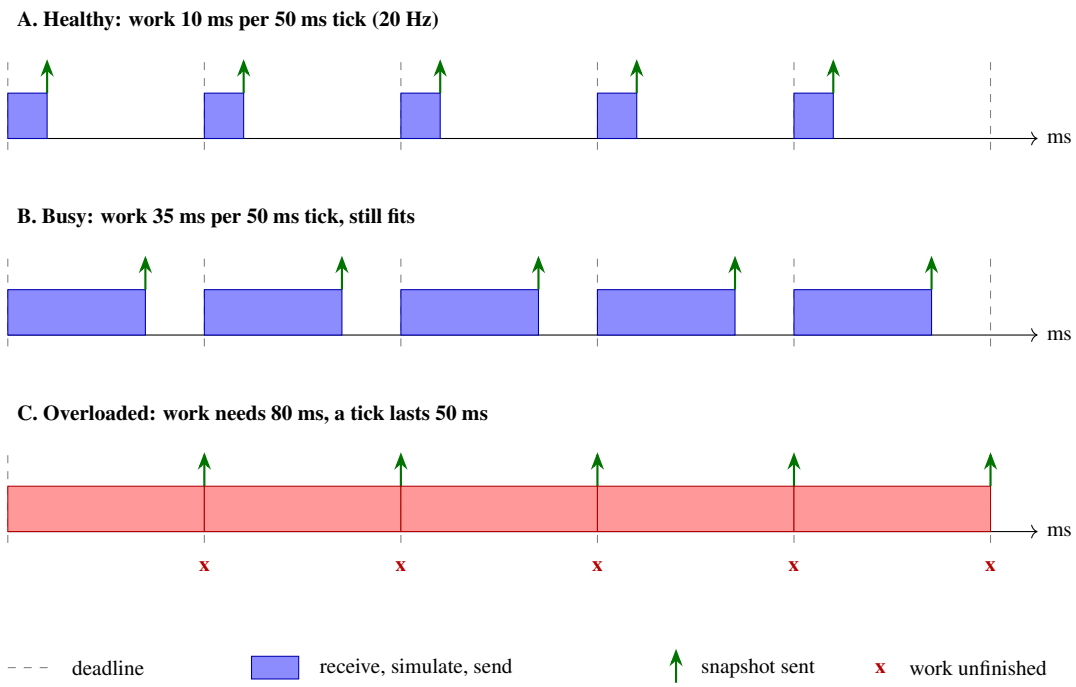


Fig. 1. Schematic of a server tick (illustrative timings, not measured data). Each tick the server receives client input, updates the world, and sends a snapshot; at 20 Hz it has 50 ms for all of it. If the work per tick grows with the number of clients and passes 50 ms (panel C), the snapshot still leaves at the 50 ms deadline, because the next tick's work has to start. The tick rate therefore stays near 20 Hz, but the work is unfinished, so the snapshots carry less up-to-date information and clients see worse ping and more gaps.

III. Making of the Godot Client and ENet Server

A. System architecture

The benchmark consists of three software components and one instrument layer. The first component is the shared Godot client, which implements both an ENet transport path using the built-in `ENetMultiplayerPeer` class and a raw UDP transport path using the `PacketPeerUDP` class. The client emits NodeStrike player state upstream and applies authoritative game state received downstream. The second component is the Godot ENet headless dedicated server, which is the same Godot project started in the server role without a window, running NodeStrike server logic and managing connections through ENet. The third component is the custom C++ UDP dedicated server, which binds a UDP socket on a configurable port, maintains a client registry indexed by IP and port, and broadcasts authoritative state to all registered clients on a fixed tick interval (Fig. 2).

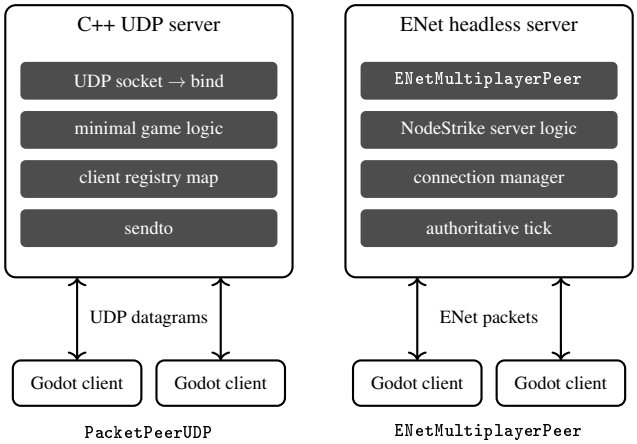


Fig. 2. The two backends under test and the shared Godot client. The same client script talks to the C++ server over raw UDP and to the Godot ENet server through ENet.

The instrumentation layer exists independently of the two backends under test. It is comprised of a per-client benchmarking logger that records packet counts, skipped snapshot numbers and ping for each test state, per-second server telemetry and per-VM CPU, memory and network use, and a Python runner that creates the Google Cloud machines and scales the population of headless Godot clients in controlled increments (Fig. 18).

B. Classes and components

TABLE II
Classes and components used by the two backends.

Classes	Side	Purpose
`ENetMultiplayerPeer`	ENet backend	ENet connection used as the multiplayer peer (server and ENet clients)
`MultiplayerAPI`	ENet backend	handles `@rpc` calls and peer connect and disconnect events
`PacketPeerUDP`	Shared client	sends and receives raw UDP packets from the Godot client
`socket`, `bind`, `recvfrom`, `sendto`	C++ backend	POSIX socket calls used to create the UDP listener, accept packets, and broadcast state
Client registry	C++ backend	in-memory map from client IP and port to player state, used to track connected clients and the time each was last heard from

C. What was built from scratch

What was made from scratch in this project:

- A C++ UDP server with a single-threaded receive loop, client registry, fixed-tick state broadcast, silence-based eviction, and a binary packet format for player state and authoritative snapshots.
- A Python runner that creates the Google Cloud machines, scales headless Godot client instances in controlled increments, and aggregates per-client metrics from each instance.
- A shared Godot client capable of switching between the ENet transport path and the raw UDP transport path with no changes to NodeStrike game logic.
- The NodeStrike demo game itself, including a grid map, bot logic for client-side movement toward a goal tile, and minimal client-side rendering for verification.

Notes from the code (delete when written).

- One Godot 4.7 project, NodeStrike (20×20 grid). One script, `main.gd` (about 860 lines), runs every role.
- The role is chosen with command-line flags: `-transport=udp|enet`, `-role=client|server`, `-address`, `-port`, `-udp-only`, `-enet-snapshot=binary|variant`.
- Client: spawns a bot, picks a goal tile, moves one tile per tick from a 20 Hz timer, and sends its position every tick (an 18-byte message over UDP, or a remote procedure call over ENet).
- Client default is the C++ UDP server; a windowed client falls back to ENet after 8 s with no reply, unless `-udp-only` is set.
- ENet server mode: creates an ENet multiplayer peer (up to 2048 clients), runs a 20 Hz timer, drops clients silent for more than 5 s, and each tick sends one snapshot to every peer by remote procedure call (compact binary, or a list of dictionaries).
- Headless mode (no window) is used for the benchmark and prints ping, packets sent and received, and gaps in snapshot numbers.
- Two small extra scripts, `enet_server.gd` and `enet_client.gd`, are a simpler ENet pair that the cloud benchmark does not use.

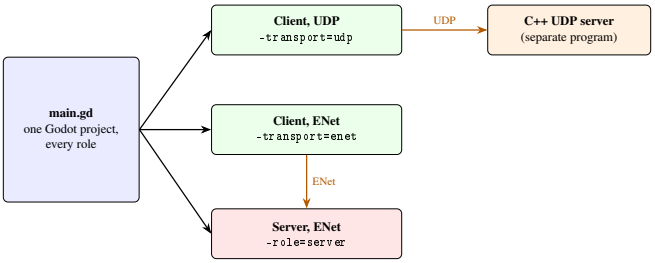


Fig. 3. The Godot project. One script is started in different modes. A UDP client talks to the separate C++ server; an ENet client talks to the same project started as an ENet server.

D. The NodeStrike workload

NodeStrike is a small grid game intended to produce a consistent and easy-to-scale network workload. Players spawn on a 20×20 grid map. Each client is a bot that picks a goal tile and moves toward it one tile per 20 Hz tick; on reaching the goal it picks a new one. The game requires only two pieces of synchronized state per tick: the position of every player and the goal tile of every player. The bot's only output is its own position and goal. This minimizes the complexity of the network behavior and simplifies the game so that it is easier to measure performance between backends rather than the game logic.

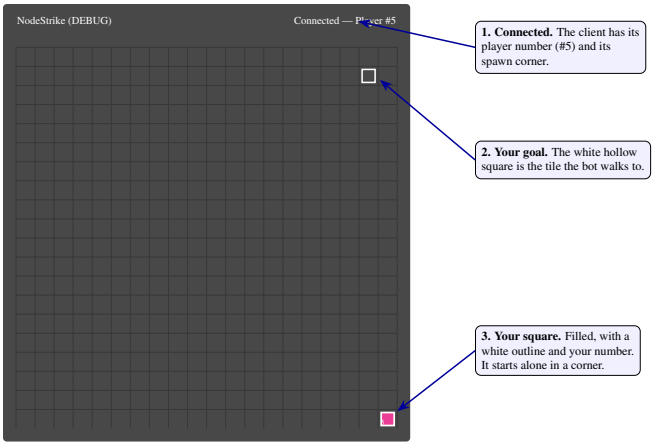


Fig. 4. The Godot client the moment it joins (a redrawing of the NodeStrike window). The board is the 20×20 grid; the only player in view is the client's own.

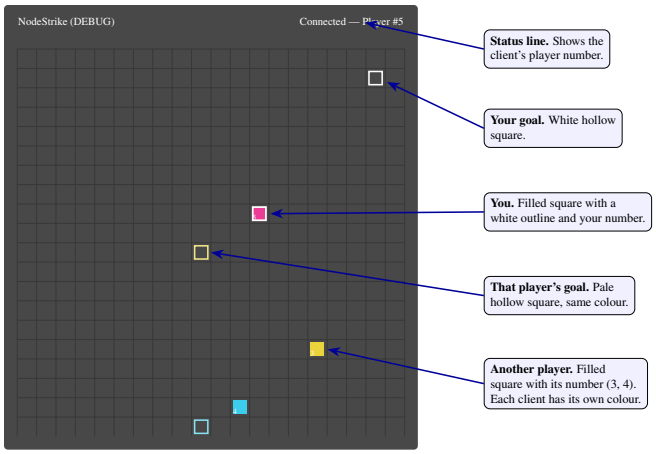


Fig. 5. The running NodeStrike client (redrawn from a screenshot of a client started in a window; headless benchmark clients draw nothing). Each tick every bot moves one tile toward its goal. The server's snapshot tells each client where every other player is; the client only draws it. Each player's pale hollow square is that player's goal.

IV. Making of the C++ UDP Server

A. Packet format

The custom C++ UDP service uses a format that defines how each packet is parsed and delivered. Each packet begins with a one-byte message type and a one-byte protocol version, followed by a payload that depends on the type of data being sent (Table III and Fig. 8). JOIN is sent by a client requesting to join, and the server answers with an ACK that carries the player id. STATE is sent by a connected client every tick with its position, a timestamp, a sequence number, and its goal tile. TICK is the snapshot that the server broadcasts to every client: a 16-byte header followed by 12 bytes (id, position and goal) for each player, split into datagrams of at most 1,200 bytes. No separate INPUT, HEARTBEAT, or DISCONNECT message is used: the STATE message sent every tick already tells the server that the client is alive, and a client that goes silent is evicted by timeout. In contrast to this custom approach, the ENet transport path utilizes Godot's native packet serialization, and its snapshot is a byte array of the same size. While this methodology diverges from the custom C++ implementation, it maintains a degree of functional equivalence sufficient for the comparative requirements of the benchmark.

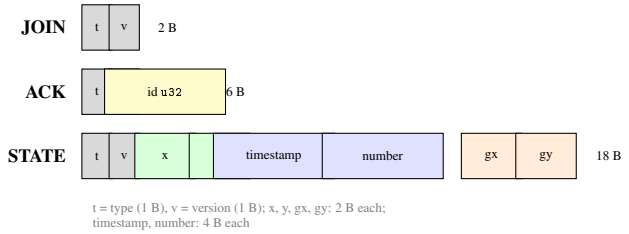


Fig. 8. Messages sent upstream (JOIN, STATE) and the acknowledgement (ACK) of the C++ server protocol. The snapshot (TICK) layout is shown in Fig. 7.

B. Heartbeat and client eviction

To address the challenge of silent disconnections, the server records a high-resolution timestamp for each client every time it receives a message from it. A client sends its state every tick (20 times per second), so this also serves as the heartbeat. Once per tick, the server compares the current system time against the last-seen timestamp of every registered client, evicting any client whose silence exceeds a predefined threshold. This timeout is set to five seconds to maintain functional equivalence with the ENet server, ensuring that connection management remains comparable across both backends (Fig. 9).

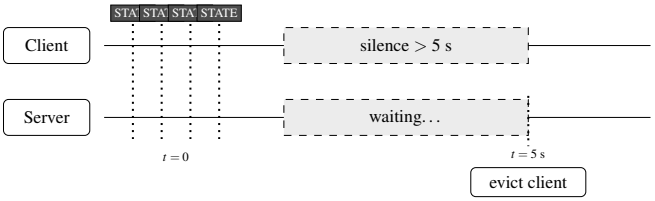


Fig. 9. Eviction by silence (schematic). Every STATE message resets the server's last-seen time for that client; a client silent for more than 5 seconds is removed.

Notes from the code (delete when written).

- One file, `server/main.cpp` (about 590 lines), C++17, plain POSIX sockets, one thread.
- Binary protocol, version 2 (Table III). A player is identified by its network address, not by anything the client sends.
- Main loop (Fig. 10): wait for a packet (until the next tick is due, at most 5 ms), read everything waiting, and when the next tick is due, run it.
- A tick drops clients silent for more than 5 s and sends a snapshot to every client, split into datagrams of at most 1,200 bytes.
- Once a second it prints one telemetry line (tick number, players, packets and bytes in and out); the benchmark records these lines.
- A separate terminal display, `tui/tui.cpp`, exists for watching a server.

TABLE III
Messages of the C++ server protocol (version 2). Snapshots larger than 1,200 bytes are split into several datagrams.

Message	Direction	Size	Contents
JOIN	client to server	2 B	type, version
ACK	server to client	6 B	type, version, player id
STATE	client to server	18 B	position, timestamp, number, goal
TICK (snapshot)	server to client	16 + 12N B	header, then id, position and goal for each of N players

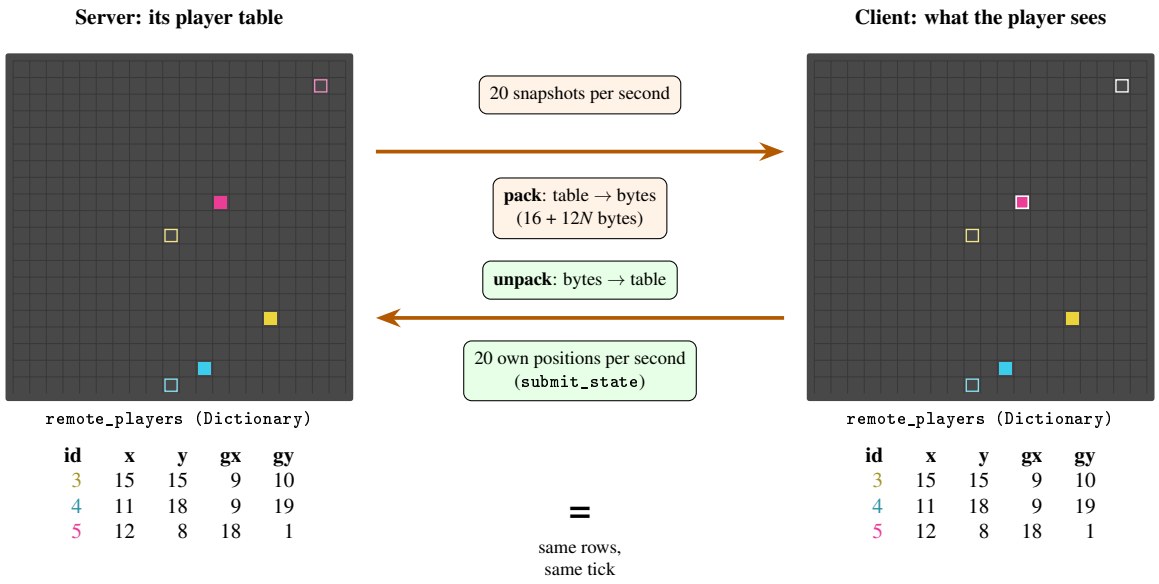
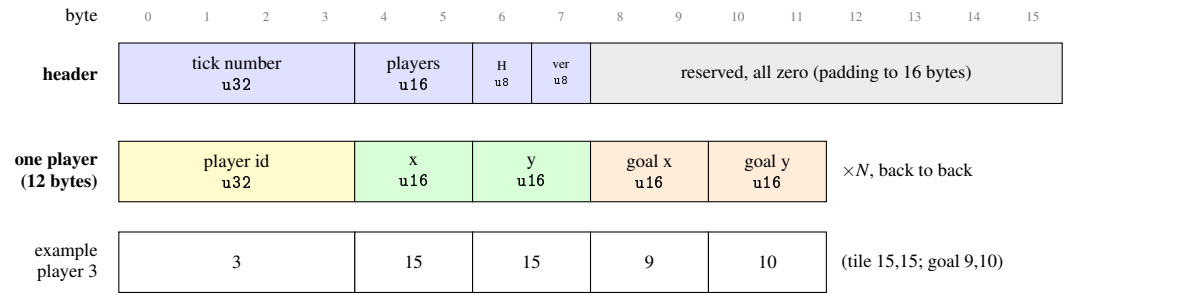
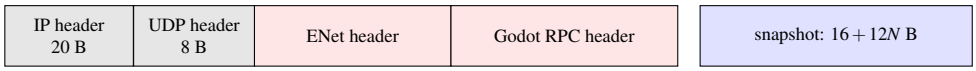


Fig. 6. Server and client side by side (redrawn). The server keeps every player in a table (a Godot Dictionary, id → position and goal). Every tick it packs the table into bytes and sends it. Each client unpacks the bytes into its own table of the same name and draws it, so the two screens show the same players and follow each other, about one network trip apart. The client's only extra is the white outline on its own square. Going the other way, each client sends only its own row.



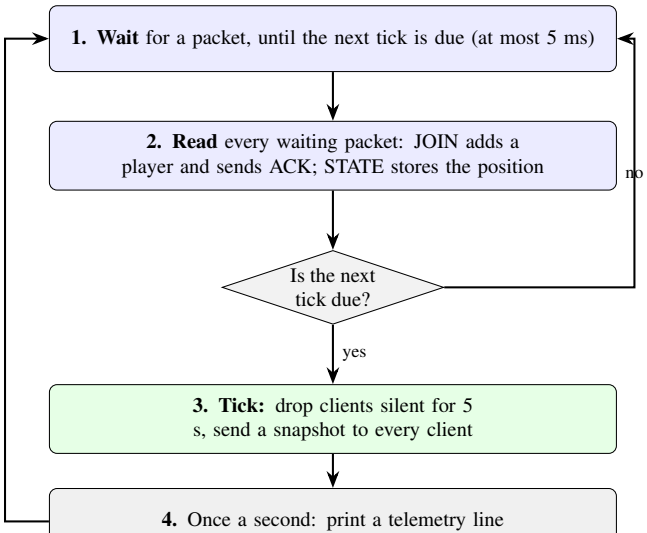
Size of the snapshot: $16 + 12N$ bytes. 3 players: 52 bytes. 300 players: 3 616 bytes. ENet splits a packet that does not fit one datagram by itself; the C++ server splits at 1 200 bytes.

What goes on the wire for one snapshot:



The ENet and Godot header sizes are not measured here. The C++ server sends the same snapshot with its own 16-byte header directly after UDP.

Fig. 7. How one snapshot is packed (ENet backend, default binary format). The server writes the header and then one 12-byte block per player into one byte array with `encode_u32`/`encode_u16` and sends it as a remote procedure call. The client reads the same positions back with `decode_u32`/`decode_u16`. The alternative `-enet-snapshot=variant` sends the same rows as a list of dictionaries, with the keys `id, x, y, gx, gy` sent as text each time.



V. How Both Servers Work: The Ins and Outs

TODO: write how the two servers work and how they differ.

A. Headless mode and engine overhead

Both servers run headless on a Google Cloud machine. In a normal Godot client, the engine runs five subsystems: rendering, audio, physics, game logic, and networking. In headless mode, rendering and audio are disabled, but physics, game logic, and networking continue to execute inside the engine process (Fig. 12). The custom C++ UDP server runs only the networking layer and the minimum game logic required to maintain authoritative state. This difference in engine overhead is precisely what the benchmark is designed to

TABLE IV
The two backends side by side.

	C++ UDP server	Godot ENet server
Networking	raw UDP datagrams	ENet through Godot's multiplayer peer
Default port	7777	7778
Tick timing	fixed-timestep schedule: each tick is due at an exact multiple of 50 ms, so it runs at 20 ticks per second (the first version restarted its timer at each tick and drifted to about 19, see Section VII)	Godot timer that fires every 50 ms and keeps its timetable (20 ticks per second)
Join	JOIN message, ACK with player id	connection handled by ENet; the peer id is the player id
Position update	18-byte STATE datagram (position, timestamp, number, goal), sent every tick	`submit_state` remote procedure call (unreliable) with the same fields, sent every tick
Snapshot	16 + 12N bytes, split into datagrams of at most 1,200 B	16 + 12N bytes (binary format, header padded from 8 to 16 bytes so both sizes match), one remote procedure call
Client eviction	silent for 5 s	silent for 5 s
Latency measure	game client's ping: last send until next snapshot	same Godot client script, same measure
Telemetry	one line per second	periodic server statistics

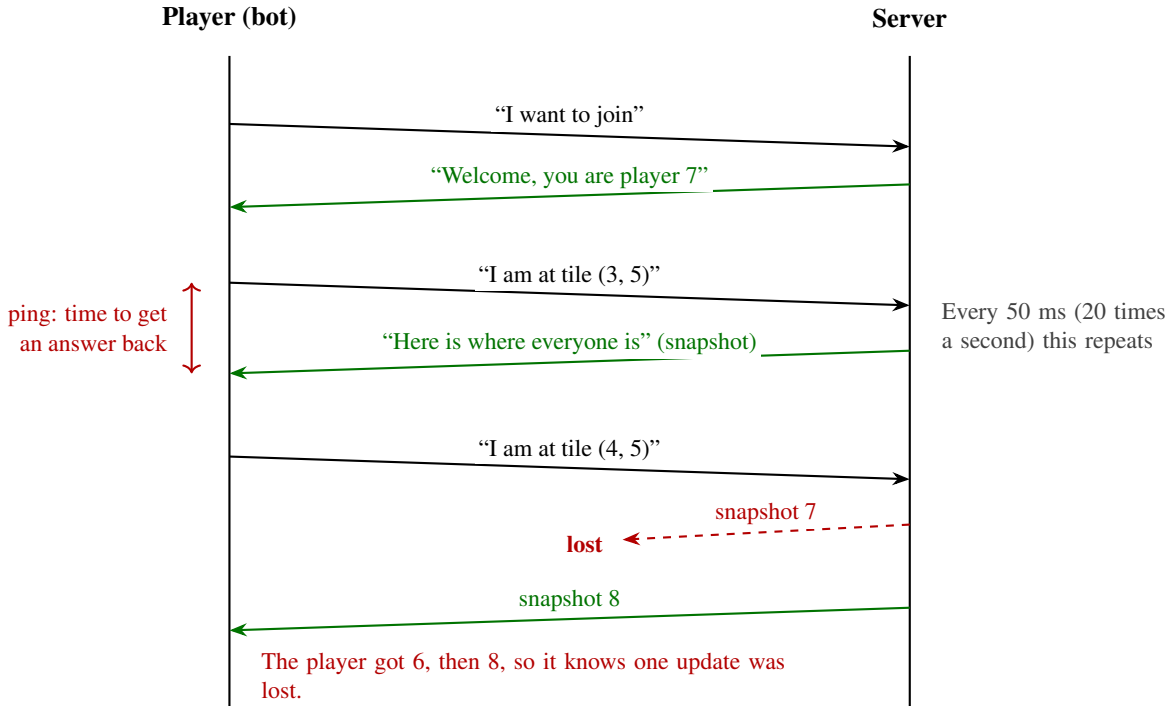


Fig. 11. What the player and the server say to each other. The player says where it is, the server replies with where everyone is, and this repeats 20 times a second. Ping is how long it takes to hear back; a lost reply shows up as a missing number. The ENet backend does the same thing through Godot's own networking.

characterize.

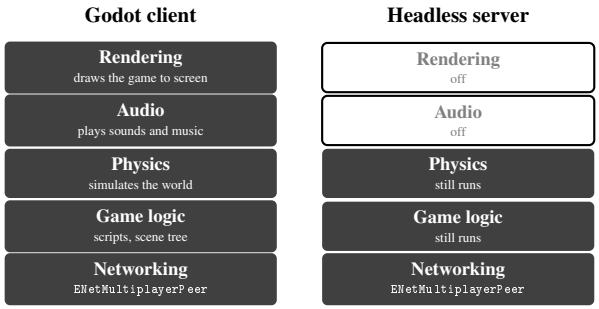


Fig. 12. What the Godot engine runs in a normal client and in headless mode. The C++ server runs only a networking layer and the minimum game logic.

B. One tick, step by step

Every 50 ms the server does the same five things, in this order. If steps 1 to 4 take less than 50 ms, it waits for the next tick. If they take longer, the next tick starts late.

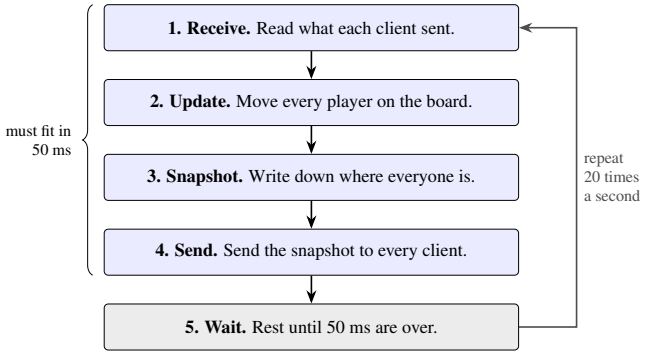


Fig. 13. The server tick. Steps 1 to 4 are the work; step 5 is spare time. The more clients there are, the longer steps 3 and 4 take.

C. Why more clients means much more work

Each snapshot lists every player, and every client gets one. So when the number of clients N doubles, each snapshot gets about twice as big and there are twice as many to send: the work grows about four times. Fig. 14 shows the amount of data this means for the C++ server (snapshot of $16 + 12N$ bytes sent to N clients, 20 times a second).

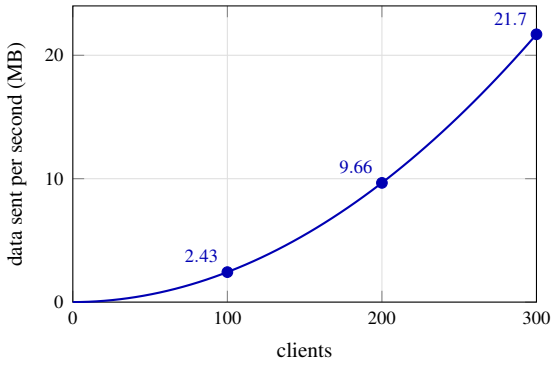


Fig. 14. Data the server must send each second as clients are added (calculated from the packet format; ENet is similar). Tripling the clients from 100 to 300 multiplies the data about nine times.

D. How clients end up responding less (and recovering)

A client counts as *responding* if it received at least one new snapshot during the measurement period. While the server keeps up, every client gets a snapshot every tick. Once the work per tick is too much, snapshots arrive late, get skipped, or pile up behind each other, and some clients stop receiving any. Fig. 15 shows the chain of events, and Fig. 16 shows what one client sees.

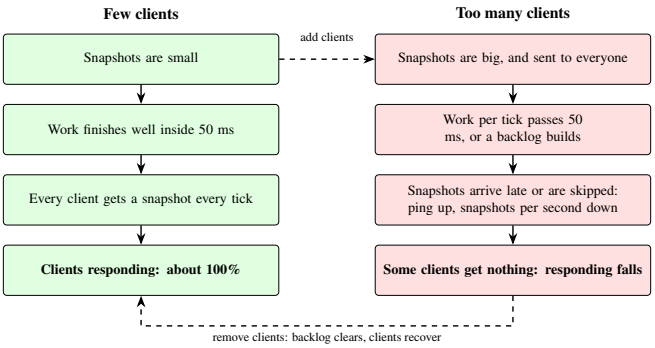


Fig. 15. Left: a server that keeps up. Right: how adding clients leads to fewer clients responding. The dashed arrows show that removing load reverses it.

VI. Experimental Method

All runs use one server VM and three client VMs on Google Cloud (the server on an e2-standard-2 machine, the clients on n2-standard-4 machines). ENet snapshots use the binary format from 22 September onward.

A. Main measures

Three measures are the main focus of the study. The rest of the data (CPU, memory, network use, server counters and every other column collected) is kept and still matters: it is used to explain why the main measures change. A tick-rate drop is the warning sign that one of the three is failing.

B. Two kinds of test

We run two kinds of test, because they answer different questions.

- **Progressive (granular) ladder.** Start with one client and add clients one step at a time, holding each step for a short time (10 s in the October runs). The state list is 1 client, then 10, 11, 12, up to 300: 292 states per backend. Question: at what client count does the server stop keeping up?
- **Hold.** Ramp up to a target client count (targets 180, 260, 270 and 300), stop adding clients, and keep them connected for 20 minutes, measured in 120 windows of 10 s. Question: does the server stay stable at a fixed load?

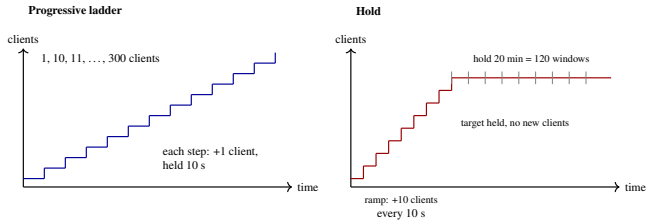


Fig. 17. The two kinds of test (schematic). The ladder keeps adding clients and shows where performance breaks. The hold stops at a target and shows whether performance stays steady.

C. How the data is collected

Fig. 18 shows where a test runs. A Python script automates every run:

1. It creates one server VM and three client VMs on Google Cloud.

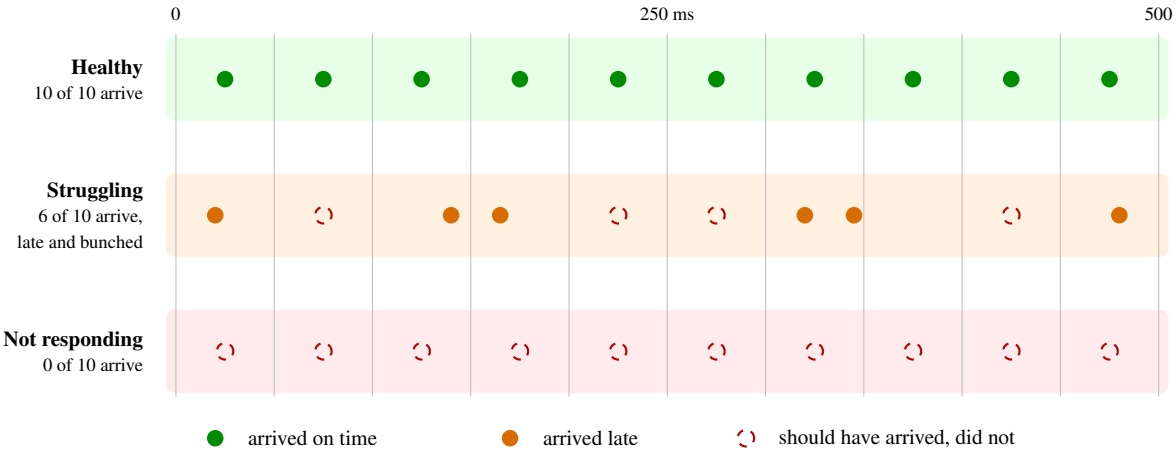


Fig. 16. What one client receives over 500 ms (schematic). Each dot is a snapshot the server should send every 50 ms. Healthy: all arrive on time. Struggling: some come late and bunched together, others never arrive. Not responding: none arrive, so the client is counted as not responding.

TABLE V
The three main measures, and the tick-rate warning sign. All other collected data supports them.

Measure	Plain meaning	On the charts
Latency	How long data takes to go to the server and back, in milliseconds (ping). Lower is better.	Ping in milliseconds (log scale)
Packet handling	How many packets the server receives and sends each second without losing or delaying them. A healthy client gets one snapshot per tick, about 20 per second.	Snapshots received per client per second (20 is perfect)
Client limit	The largest number of clients the server supports before performance degrades.	Share of clients still responding; where the lines break
Tick rate (warning sign)	Ticks the server completes per second. 20 Hz is healthy; below 18 Hz means the server is falling behind.	Server ticks per second, with an 18 Hz line

2. It starts the server (the C++ UDP server, or the Godot ENet server in headless mode).
3. It gives each client VM a plan: a list of states, each with a total client target and a duration. The client VMs start bot clients so that the total matches the plan, and each bot plays the Node-Strike game. Every bot is the same Godot client (`main.gd`): in UDP mode it talks to the C++ server and in ENet mode to the Godot server, so both servers are driven by the same client code.
4. During every state, each client counts what it sent and received, checks the snapshot sequence numbers for gaps, and measures ping about once per second.
5. The server writes one telemetry row per second (tick count, players connected, packets and bytes in and out). Each VM also records CPU, memory and network use every second.
6. When the run ends, all files are downloaded to this computer, the VMs are deleted, and analysis scripts merge the repetitions and apply pass/fail thresholds.

Gaps and incomplete snapshots are indicators of protocol-level loss, not measured network packet-loss percentages.

D. What the charts show

Every chart plots the same four things for both backends:

- **Latency:** average ping.
- **Packet handling:** snapshots received per client per second.
- **Clients responding:** how many clients are still getting answers.
- **Tick rate:** server ticks per second.

On the ladder charts the horizontal axis is the number of clients, so each point is one client count. On the hold charts it is time, so each point is one 10 s window.

E. Health criteria and caveats

TODO: write the pass/fail limits and what the measurements can and cannot show.

F. Precision limits

TODO: write the precision limits.

G. Runs

Granular ramps with $N_{\max} = 300$ give 292 states per backend (1 plus 10 through 300).

VII. Problems Found in the First Runs, and How They Were Fixed

After the first complete set of runs we found two problems that made the comparison between the C++ and the ENet server unfair, plus some measurement quirks. This section explains, for each one, what

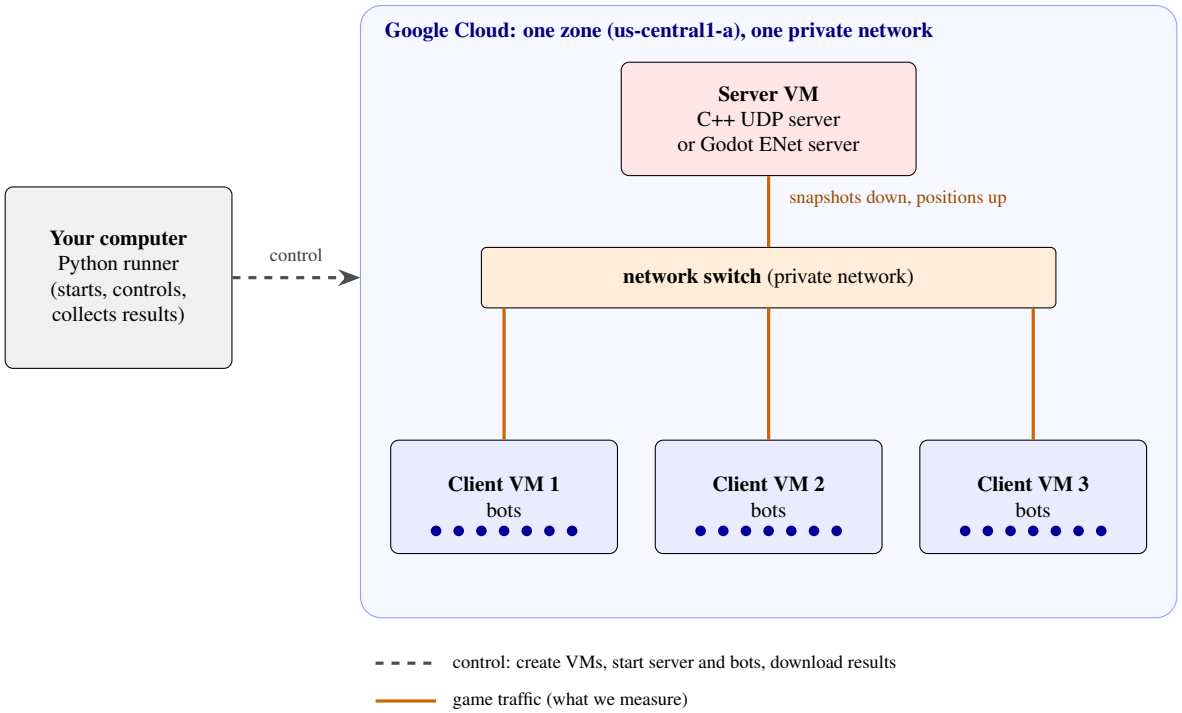


Fig. 18. Where a test runs. The runner on your computer creates one server VM and three client VMs, starts the server, tells each client VM how many bots to run, and downloads the results afterwards. The bots and the server talk to each other over the cloud's private network (a switch). Each client VM runs about a third of the bots, so the server sees all of them as one crowd.

TABLE VI
Granularity of each measured quantity. The precision limit of a conclusion is set by the coarsest quantity it relies on.

Quantity	Granularity	Where the limit comes from
Ping proxy	1 ms	The client logs whole milliseconds, but a snapshot only arrives every 50 ms (20 Hz), so differences of a few ms between backends are not reliable (limiting quantity).
Server tick rate	0.01 Hz	Computed from 9–10 samples in each 10 s window.
Snapshots per client per second	0.01	Count over a 10 s window divided by the clients.
Packet gaps	1 snapshot	Integer count of skipped snapshot numbers.
Clients responding	1 client	Whole clients.
CPU	0.1 %	Sampled once per second and averaged over the window.

TABLE VII
Runs used in the results. The progressive ladder is eight complete runs per backend, each run a separate pass from 1 to 300 clients (one repetition each). The C++ server is driven by Godot UDP clients and the ENet server by Godot ENet clients. The hold tests are eight runs per backend and target, each a 20-minute hold (one repetition each) after a ramp-up of 10 clients per 10 s.

Date (2026)	Type	Backend	Counts	Time/state	Runs	Status
Oct 23–24	Progressive ladder (granular)	C++ server, ENet server	1, 10–300	10 s	8 runs	ok
Oct 24	Hold	C++ server, ENet server	180, 260, 270, 300	20 min hold	8 runs	ok

it was, why it happened, how it was fixed, and what it means for the first-round data. The first-round results in Section VIII are kept for reference and are superseded: the corrected data (8 progressive-ladder runs and 8 hold runs per backend, with the patches below applied) is in Section VIII.

A. Overview

B. Problem 1: the C++ tick drifted below 20 Hz

1) What we saw

Both servers are meant to tick at 20 Hz. In the first round the ENet server ticked at about 20 Hz, but the C++ server ticked at about 19.4 Hz with few clients and fell to about 18.9 Hz at 300 clients. Over a second that is roughly 5% fewer ticks and snapshots, so C++ was doing slightly less work than ENet.

2) Why it happened: the stopwatch

The C++ loop starts a tick as soon as it notices that 50 ms have passed, and then restarts its timer from that moment. If it notices 2 ms late, the next 50 ms are counted from the late moment, so the delay is never made up and each tick is 52 ms after the last. In one second that is $1000/52 \approx 19.2$ ticks instead of 20. With more clients each pass through the loop does more work, so the check for the next tick comes later and the drift grows. In the measured data the

TABLE VIII
The problems found in the first runs.

Problem	Why it mattered	Fix	Status
C++ ticks drifted to about 19 Hz	C++ did about 5% fewer ticks per second than ENet	Fixed-timestep tick schedule	Fixed
Snapshot sizes differed (8 vs 16 header bytes)	ENet sent slightly smaller packets than C++	ENet header padded to 16 bytes	Fixed
Slow ENet states and a last-state artifact	ENet rates were measured over longer windows; the last ladder state was inflated	To be handled in the analysis	Noted

tick falls from about 51.5 ms (about 1.5 ms late) with few clients to about 52.9 ms (about 2.9 ms late) at 300 clients. The ENet server uses a Godot timer, which keeps its timetable, so it does not drift.

3) The drift in one picture

Fig. 19 counts the ticks of both servers side by side. Each dot is one tick and the number is the tick count.

4) A worked example

Say every tick is noticed 2 ms late.

- **Stopwatch (C++ as first built).** Ticks land at 0, 52, 104, 156, ... ms. After 10 ticks the server is 20 ms behind the ideal schedule, and after one second it has completed only 19 ticks.
- **Timetable (ENet, and the C++ fix).** Ticks are aimed at 50, 100, 150, ... ms. A late tick is followed by an on-time one, so after one second exactly 20 ticks have been completed.

5) The code behind it

Fig. 20 shows the real tick check from the first version of the C++ server and Fig. 21 explains it in pseudocode. The line that causes the drift is `last_tick = now;`.

```
// first version (server/main.cpp): the tick loop
auto   now     = Clock::now();
double elapsed = std::chrono::duration<double>(now -
    last_tick).count();
if (elapsed >= TICK_SEC) {      // 50 ms or more have
    passed
    last_tick = now;            // restart the timer at "
    now": the delay is lost
    tick_n++;
    // ... drop silent clients, broadcast snapshots ...
}

// fix (applied): advance the timer by exactly one tick
last_tick += std::chrono::duration_cast<Clock::duration>(
                 std::chrono::duration<double>(TICK_SEC))
    ;
```

Fig. 20. The tick check in the C++ server and the one-line change that removes the drift.

6) The fix

Two changes were made in `server/main.cpp`. First, the next tick is scheduled at an exact multiple of 50 ms after the previous due time, so lateness does not add up. Second, the wait for packets now ends when the next tick is due (at most 5 ms), so ticks are noticed sooner. If the server ever falls more than a whole tick behind, it resynchronizes instead of firing a burst of ticks. Fig. 22 shows the real code, including the resynchronization check.

```
// fixed code (server/main.cpp, excerpts)

// before the loop: the tick schedule
const auto tick_period = std::chrono::duration_cast<Clock
    ::duration>(
                             std::chrono::duration<double
    >(TICK_SEC));
auto next_tick = Clock::now() + tick_period;

while (true) {
    // sleep until the next tick is due (at most 5 ms) so
     ticks are not late
    auto until_tick_us = std::chrono::duration_cast<std::
    chrono::microseconds>(
        next_tick - Clock::now()).count();
    if (until_tick_us < 0)    until_tick_us = 0;
    if (until_tick_us > 5000) until_tick_us = 5000;
    // ... wait for packets for until_tick_us, then read
    every waiting packet ...

    // 20 Hz tick
    auto now = Clock::now();
    if (now >= next_tick) {
        next_tick += tick_period;
        if (next_tick <= now)            // fell more
    than a tick behind: resync
            next_tick = now + tick_period;
        // ... tick: drop silent clients, broadcast
    snapshots ...
    }
}
```

Fig. 22. The fixed-timestep tick schedule in the C++ server (excerpts from `server/main.cpp`). The key change is `next_tick += tick_period`: the next tick is due one period after the previous due time, not after the moment the tick was noticed.

7) Result of the fix

In a 30-second local test the original server ticked at 19.67 Hz and the fixed server at 20.000 Hz. How this changes the cloud results will be reported with the corrected runs.

C. Problem 2: snapshots of different size

Both servers send every client a snapshot of all players, 12 bytes per player, plus a header. The C++ header was 16 bytes and the ENet binary header only 8, so each C++ snapshot was 8 bytes larger. The difference is tiny (Table IX) but it is an unnecessary difference between the two backends, so the ENet header was padded to 16 bytes (bytes 8 to 15 are reserved and zero; the first 8 bytes are unchanged). Both transports now send 16 + 12N bytes.

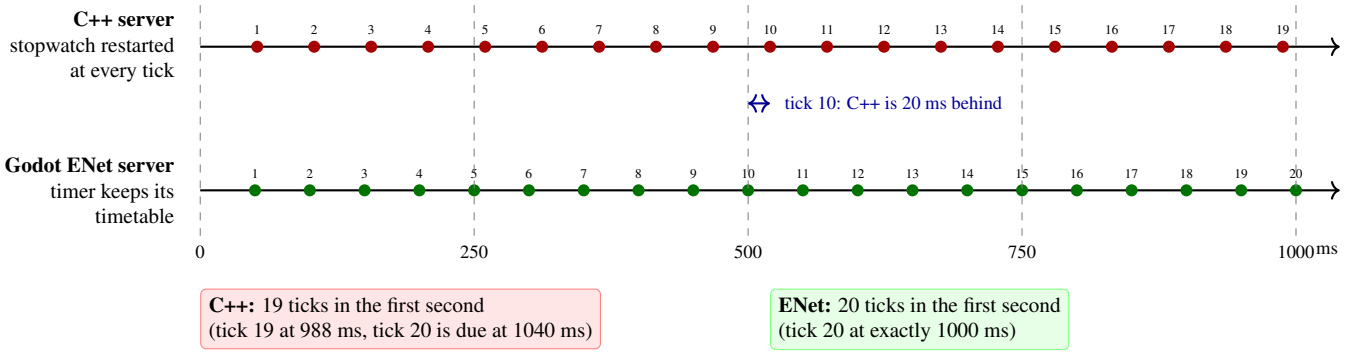


Fig. 19. Counting ticks, C++ against ENet (schematic, each tick noticed 2 ms late). Every dot is one tick and the number is the tick count. The C++ loop restarts its stopwatch at each tick, so its ticks are 52 ms apart and it has counted only 19 ticks after one second. The Godot timer keeps its 50 ms timetable, so its tick 20 arrives at exactly 1000 ms. After 10 ticks the C++ server is already 20 ms behind, and the gap keeps growing.

```
STOPWATCH (first C++ version)        TIMETABLE (Godot timer / the fix)
------------------------------       ---------------------------------
every loop:                          every loop:
  if now - last_tick >= 50 ms:         if now - last_tick >= 50 ms:
    last_tick = now                      last_tick = last_tick + 50 ms
    do the tick                          do the tick

Stopwatch: noticed at 52 ms -> next tick due at 52 + 50 = 102 ms (drifts).
Timetable: noticed at 52 ms -> next tick due at 50 + 50 = 100 ms (on schedule).
```

Fig. 21. Pseudocode for the two ways of timing a tick. Think of a bus that should leave every 50 minutes: the stopwatch driver restarts his watch whenever he notices the bus is due, so any lateness is kept; the timetable driver follows fixed times, so lateness is not kept.

TABLE IX
Size of one snapshot sent to one client.

Players N	C++ (bytes)	ENet before (bytes)	ENet now (bytes)
100	1,216	1,208	1,216
200	2,416	2,408	2,416
300	3,616	3,608	3,616

D. Measurement quirks noticed

- **Slow ENet states.** When ENet was overloaded, a "10 s" ladder state actually lasted about 27 s, because the harness waited for slow client logs. Rates are computed over the real length, so they are correct.
- **The last ladder state.** The last state (300 clients) has no later state to measure its length from, so the nominal 10 s was used while its counters covered about 27 s. This inflated its snapshot rate (up to about 30 per second, which cannot be real at 20 ticks per second).

The last ladder state is left out of the corrected ladder plots and tables (Section VIII-A).

E. What changes in the corrected runs

- The C++ server ticks at 20 Hz (fixed-timestep schedule).
- Both servers send 16 + 12N byte snapshots.
- 8 progressive-ladder runs and 8 hold runs per backend were made again with both patches applied.

Status (31 October 2026): both fixes are applied, and the corrected data (8 progressive-ladder runs and 8 hold runs per backend) is in Section VIII; only the first-round example data is still superseded.

VIII. Results

A. *Test 1, progressive ladder: granular runs (both patches applied)*

TODO: interpret the ladder results (latency, packet gaps, client limit, tick rate).

B. *Test 2, hold at 300 clients*

TODO: interpret the 300-client hold.

C. *Test 2, hold at every target*

TODO: interpret the holds at every target.

D. *[SUPERSEDED] Example data*

SUPERSEDED. First-round data, kept for reference. Replaced by the granular runs in Section VIII-A; the hold tests will be redone with the corrected servers.

The tables below are real rows from our runs, to show what the data looks like.

Progressive ladder: 8 runs summed up. Each row sums up the 8 complete ladder runs at one client count: the mean, with the lowest and highest run in brackets. Resp. = clients responding (mean), Snap/s = snapshots received per client per second, Tick = server ticks per second, Passing = runs that met the health limits at that count.

One thing to note. The ENet server's tick rate stays near 20 Hz even when its clients are failing, so for ENet the breakdown shows up as lost clients, low snapshot rate and high ping rather than a tick-rate drop. TODO: confirm why. The ENet server counts ticks from a Godot timer that fires every 50 ms without drift, so its tick counter keeps pace with the clock whether or not the work finished in time.

E. *How the runs were summed up*

Both tests combine 8 complete runs per backend. For the ladder, each of the 292 client counts is paired with the same count in the other runs. For the holds, each 10 s state is paired by its position in the ramp-up or hold. The mean, lowest and highest value of each measure are reported. A ladder run passes at a client count if that count met the health limits; a hold run held if its hold averages met them. Hold run 5 needed a second attempt; its first attempt is not used.

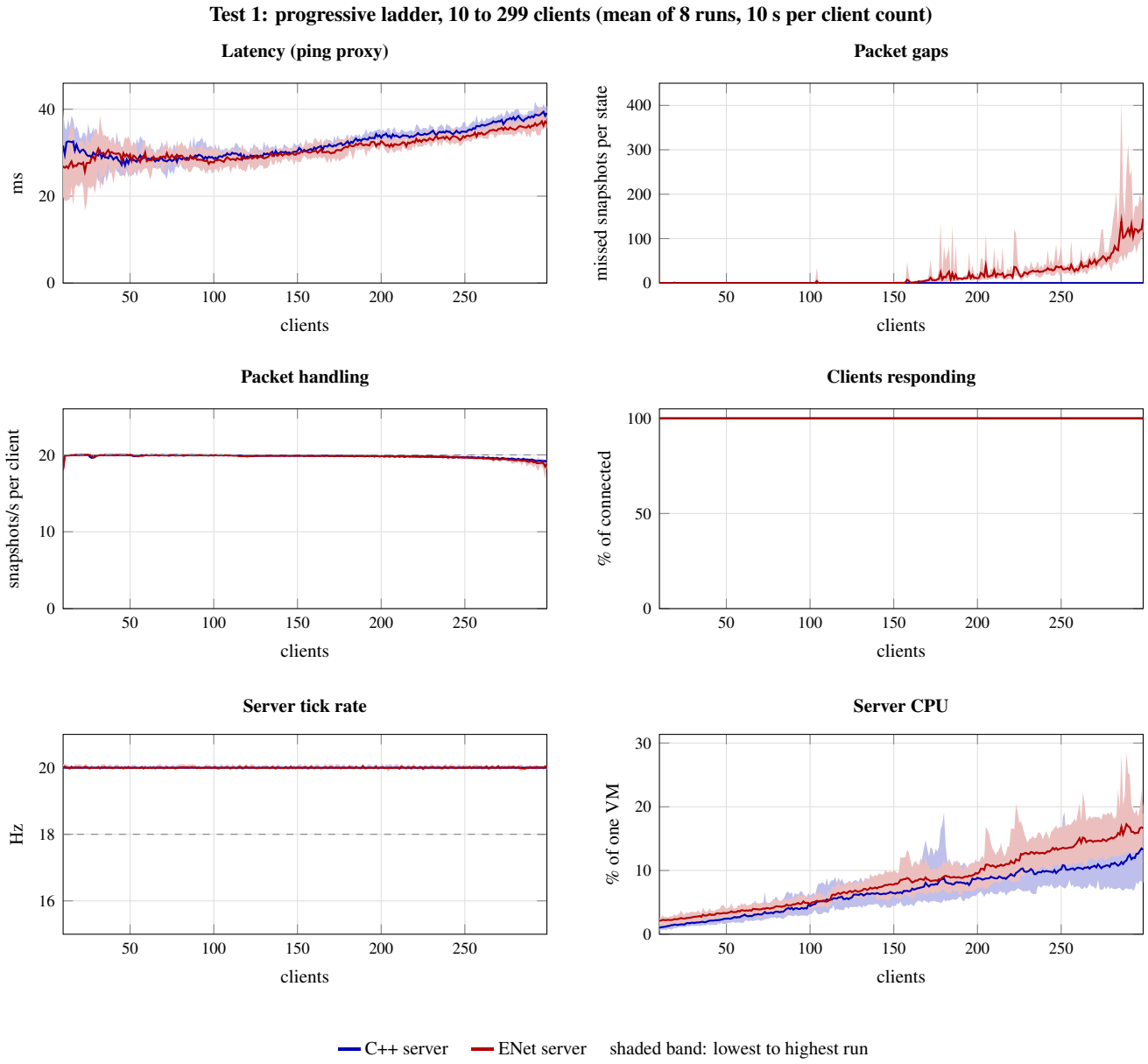


Fig. 23. Test 1, progressive ladder (granular batch, both patches applied; server on an e2-standard-2 VM, three n2-standard-4 client VMs, 10 s per client count; the 1-client warm-up state and the last state, 300 clients, are left out: the last state has no later state to time it by, so its counters cover a longer period than 10 s, see Section VII). Each line is the mean of 8 runs and the band spans the lowest to the highest run. The C++ server is driven by Godot UDP clients and the ENet server by Godot ENet clients. Ping is measured by the Godot client (time from its last send until a snapshot arrives), not a true round trip. Packet gaps are skipped snapshot numbers summed over all clients in a state. Dashed lines: 20 snapshots per second and the 18 Hz tick threshold.

TABLE X

Ladder results at selected client counts, summed up over 8 runs (10 s per state). Ping, Snap/s and Gaps show the mean and, in brackets, the lowest and highest run. Resp. = clients responding (mean); Ping = the Godot client's ping proxy; Snap/s = snapshots received per client per second; Gaps = skipped snapshot numbers in that state; Tick = server ticks per second; Srv CPU = server CPU.

Backend	Clients	Resp. (%)	Ping, ms	Snap/s	Gaps	Tick (Hz)	Srv CPU (%)
C++ server	100	100.0	29 (26–31)	19.9 (19.9–20.0)	0 (0–0)	20.00	4.4
C++ server	150	100.0	30 (29–32)	19.9 (19.8–19.9)	0 (0–0)	20.00	6.4
C++ server	200	100.0	33 (32–35)	19.8 (19.7–19.9)	0 (0–0)	20.00	8.9
C++ server	250	100.0	35 (33–37)	19.6 (19.5–19.9)	0 (0–0)	20.00	10.2
C++ server	299	100.0	39 (38–41)	19.0 (18.8–19.3)	0 (0–0)	20.00	13.3
ENet server	100	100.0	28 (25–30)	19.9 (19.9–20.0)	0 (0–0)	20.01	4.7
ENet server	150	100.0	30 (28–31)	19.9 (19.9–19.9)	0 (0–1)	20.01	7.8
ENet server	200	100.0	33 (32–34)	19.8 (19.8–19.9)	10 (7–16)	20.00	9.5
ENet server	250	100.0	33 (33–34)	19.7 (19.5–19.8)	38 (20–85)	20.00	13.5
ENet server	299	100.0	37 (36–40)	18.9 (18.4–19.1)	145 (116–201)	19.99	16.6

TABLE XI

Per run, over the ladder (291 states, 1 and 10 to 299 clients): states that met the health limits, snapshot gaps summed over all states, and the first client count (from 10 upward) at which a state failed (– = none).

	C++ server			ENet server		
Run	Passed	Gaps	First fail	Passed	Gaps	First fail
1	291	0	–	290	4,257	–
2	291	0	–	290	3,807	–
3	290	0	–	287	5,367	294
4	290	0	–	290	6,951	–
5	290	0	–	291	4,722	–
6	290	0	–	290	4,267	–
7	291	0	–	290	5,847	–
8	290	5	–	291	4,710	–

TABLE XII

Hold results at each target, from the 8 runs (C++ server with Godot clients; ENet server with ENet clients). Each run is first averaged over its 20-minute hold. Ping shows the median of the 8 run averages with the standard deviation in brackets. Gaps = skipped snapshot numbers summed over all clients and the whole hold: median with the lowest and highest run in brackets; "% of exp." is the median divided by the snapshots expected (clients $\times$ 20 per second $\times$ 1200 s). Cert. = certainty (percent) from the 8 run values: one minus the normalized fractional uncertainty (coefficient of variation divided by $\sqrt{8}$) raised to the scaling factor 0.4; n/a where the median is 0. Tick err. = absolute fractional error of the mean server tick rate against the 20 Hz baseline. Every run at every target responded at 100% and received 20 snapshots per client per second (19.96 to 20.00). A run "held" if its hold averages met the health limits (at least 99% responding, at least 18 snapshots per second, ping at most 150 ms, tick at least 18 Hz).

Server	Clients	Ping, ms	Ping cert. (%)	Gaps (20 min)	Gaps (% of exp.)	Gaps cert. (%)	Tick err. (%)	Held
C++ server	180	32.1 (0.2)	91	0 (0–1)	0.0000	n/a	0.00	8 of 8
C++ server	260	36.2 (0.3)	90	0 (0–1175)	0.0000	n/a	0.00	8 of 8
C++ server	270	37.1 (0.3)	91	0 (0–0)	0.0000	n/a	0.00	8 of 8
C++ server	300	39.1 (0.4)	90	0 (0–8)	0.0000	n/a	0.00	8 of 8
ENet server	180	31.4 (0.5)	87	465 (353–830)	0.0108	59	0.01	8 of 8
ENet server	260	34.4 (0.4)	89	3372 (3004–3771)	0.0540	76	0.00	8 of 8
ENet server	270	34.9 (0.2)	91	4756 (3867–5031)	0.0734	76	0.00	8 of 8
ENet server	300	36.9 (0.4)	89	11391 (9659–14963)	0.1582	70	0.00	8 of 8

TABLE XIII

[Superseded] Example ladder rows, summed up over 8 runs (16–17 October, 10 s per state). Ping and Snap/s show the mean and, in brackets, the lowest and highest run.

Backend	Clients	Resp. (%)	Ping, ms	Snap/s	Tick (Hz)	Passing
Godot ENet	100	100.0	29 (28–31)	19.9 (19.8–19.9)	20.0	8 of 8
Godot ENet	150	100.0	33 (31–34)	19.6 (19.3–19.9)	20.0	8 of 8
Godot ENet	200	100.0	38 (34–42)	18.0 (16.5–20.2)	20.0	3 of 8
Godot ENet	250	100.0	478 (46–942)	14.4 (12.8–17.6)	20.0	0 of 8
Godot ENet	300	99.3	252 (50–1474)	31.2 (28.2–35.8)	20.0	3 of 8

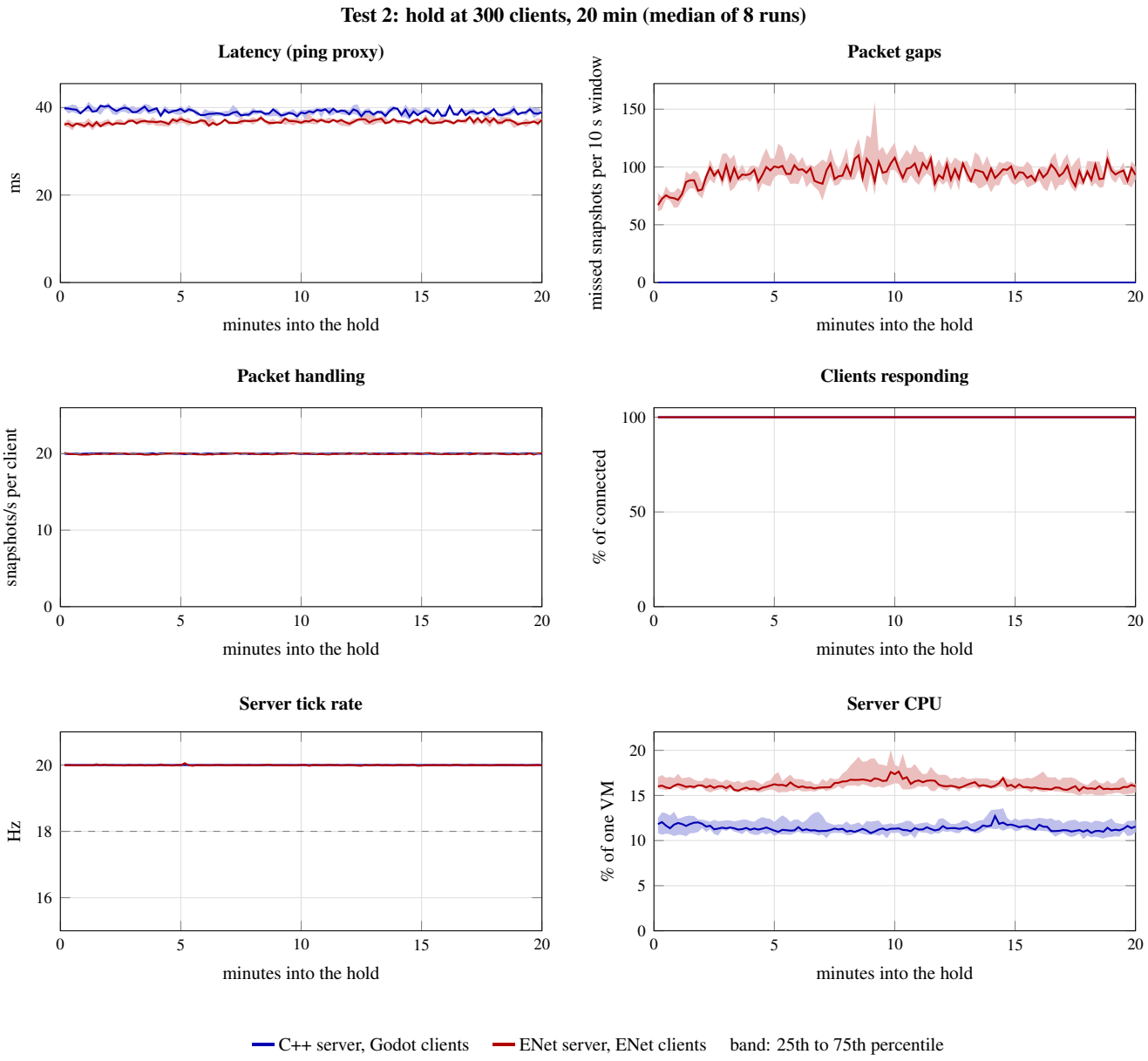


Fig. 24. Test 2, hold at 300 clients (server on an e2-standard-2 VM, three n2-standard-4 client VMs). The clients are added first (ramp-up, 10 clients every 10 s) and the plot starts when the last client has joined; it then covers the 20-minute hold with no new clients. Each line is the median of 8 runs and the band spans the 25th to the 75th percentile. Packet gaps are skipped snapshot numbers summed over all clients in a 10 s window. Ping is the Godot client's ping proxy, not a true round trip. Dashed lines: 20 snapshots per second and the 18 Hz tick threshold.

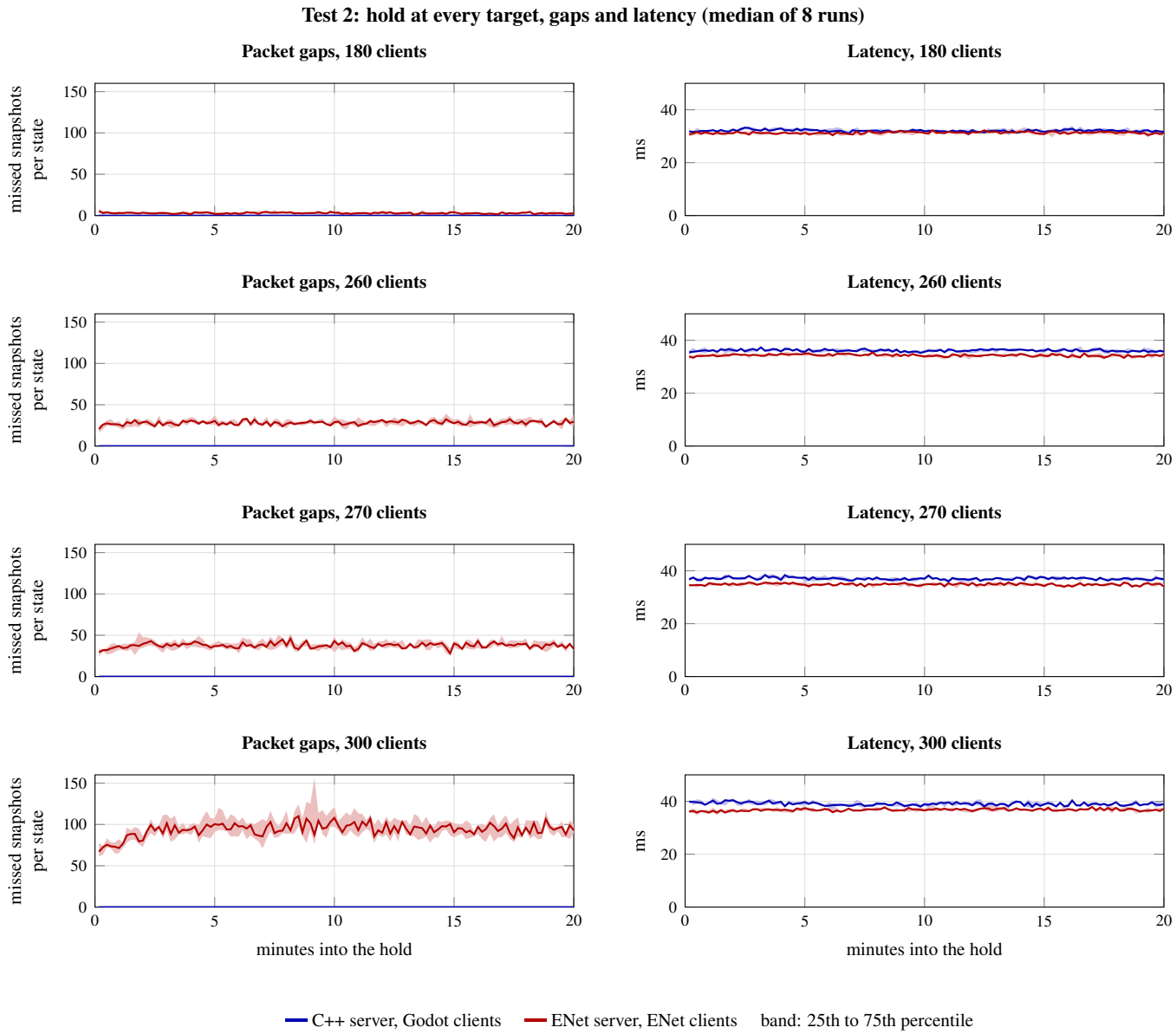


Fig. 25. Test 2, packet gaps (left) and latency (right) during the 20-minute hold at 180, 260, 270 and 300 clients. Each line is the median of 8 runs and the band spans the 25th to the 75th percentile. Gaps are skipped snapshot numbers summed over all clients in a 10 s window.

IX. Discussion

TODO.

X. Conclusion

TODO.

References

[1] L. Aukstakalnis, "Server-client networking implementation using the Godot game engine," 2024.

[2] Godot Foundation, "UDPServer," *Godot Engine Documentation*, 2026. [Online]. Available: `https://docs.godotengine.org/en/4.4/classes/class_udpserver.html`. Accessed: Feb. 27, 2026.

[3] Godot Foundation, "Exporting for dedicated servers," *Godot Engine Documentation*, 2026. [Online]. Available: `https://docs.godotengine.org/en/stable/tutorials/export/exporting_for_dedicated_servers.html`. Accessed: Feb. 27, 2026.

[4] F. T. Al-Dhief *et al.*, "Performance comparison between TCP and UDP protocols in different simulation scenarios," *Int. J. Eng. Technol.*, vol. 7, no. 4.36, pp. 172–176, 2018.

[5] C. Jones, "Go UDP programming: A beginner-friendly guide to building fast, real-time apps," *DEV Community*, Oct. 29, 2025. [Online]. Available: `https://dev.to/jones_charles_ad50858dbc0/go-udp-programming-a-beginner-friendly-guide-to-building-fast-real-time-apps-4ik`. Accessed: Feb. 27, 2026.

[6] E. Retunsky, "Benchmarking low-level I/O: C, C++, Rust, Golang, Java, Python," *Star Gazers*, Mar. 26, 2023. [Online]. Available: `https://medium.com/star-gazers/benchmarking-low-level-i-o-c-c-rust-golang-java-python-9a0d505f85f7`

[7] F. J. M. Arboleda, M. R. Arias, and J. A. H. Riveros, "Performance of parallelism in Python and C++," *IAENG Int. J. Comput. Sci.*, vol. 50, no. 2, pp. 1–13, 2023.

[8] F. Zehra, M. Javed, D. Khan, and M. Pasha, "Comparative analysis of C++ and Python in terms of memory and time," *Preprints*, 2020, doi: 10.20944/preprints202012.0516.v1.

[9] GeeksforGeeks, "UDP server-client implementation in C++," Mar. 22, 2018. [Online]. Available: `https://www.geeksforgeeks.org/cpp/udp-server-client-implementation-c/`

[10] A. D. Çelik and G. Şeçinti, "Network optimizing software solution for multiplayer gaming," *ITU J. Wireless Commun. Cybersecurity*, vol. 1, no. 1, pp. 47–56, 2024.

[11] J. Ahde, "Real-time Unity multiplayer server implementation," B.S. thesis, 2017.

[12] J. Gettys and K. Nichols, "Bufferbloat: Dark buffers in the Internet," *ACM Queue*, vol. 9, no. 11, 2011.

[13] Unity Technologies, "Ticks and update rates," *Netcode for GameObjects 2.5 Manual*. [Online]. Available: `https://docs.unity3d.com/Packages/com.unity.netcode.gameobjects@2.5/manual/learn/ticks-and-update-rates.html`

[14] Valve Developer Community, "Source multiplayer networking." [Online]. Available: `https://developer.valvesoftware.com/wiki/Source_Multiplayer_Networking`

[15] M. Claypool and K. Claypool, "Latency and player actions in online games," *Commun. ACM*, vol. 49, no. 11, pp. 40–45, Nov. 2006.

[16] Y. W. Bernier, "Latency compensating methods in client/server in-game protocol design and optimization," presented at the Game Developers Conf., 2001.

[17] J. Postel, "User datagram protocol," IETF, RFC 768, Aug. 1980.

[18] L. Eggert, G. Fairhurst, and G. Shepherd, "UDP usage guidelines," IETF, RFC 8085, Mar. 2017.